



Dominando el Control de Versiones

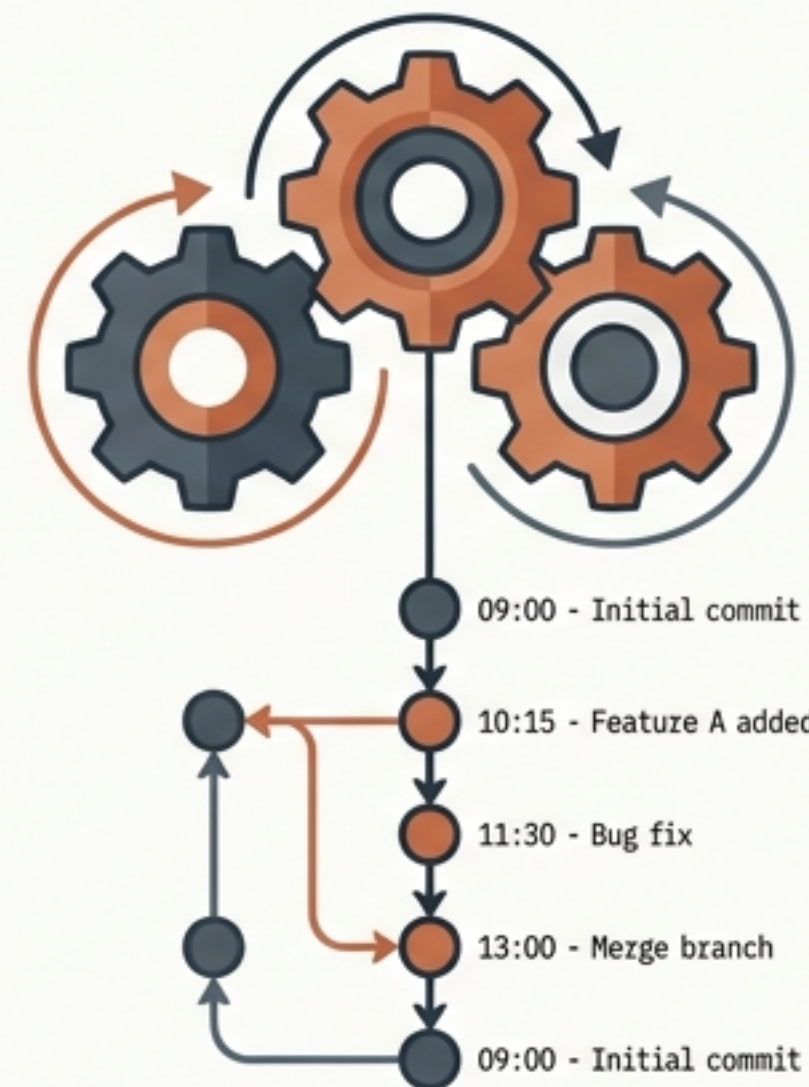
Arquitecturas, herramientas y flujos de trabajo colaborativo para equipos de desarrollo.

>_ Fuente: Tema 19. Entornos de desarrollo

El Desafío de la Colaboración Sincronizada



Sin un sistema de control, el trabajo en equipo genera sobrescrituras accidentales, pérdida de código y un historial de cambios indescifrable.



Un Sistema de Control de Versiones (VCS) centraliza la sincronización, registra el historial cronológico de cada modificación y aumenta radicalmente la productividad del equipo al permitir un desarrollo paralelo seguro.

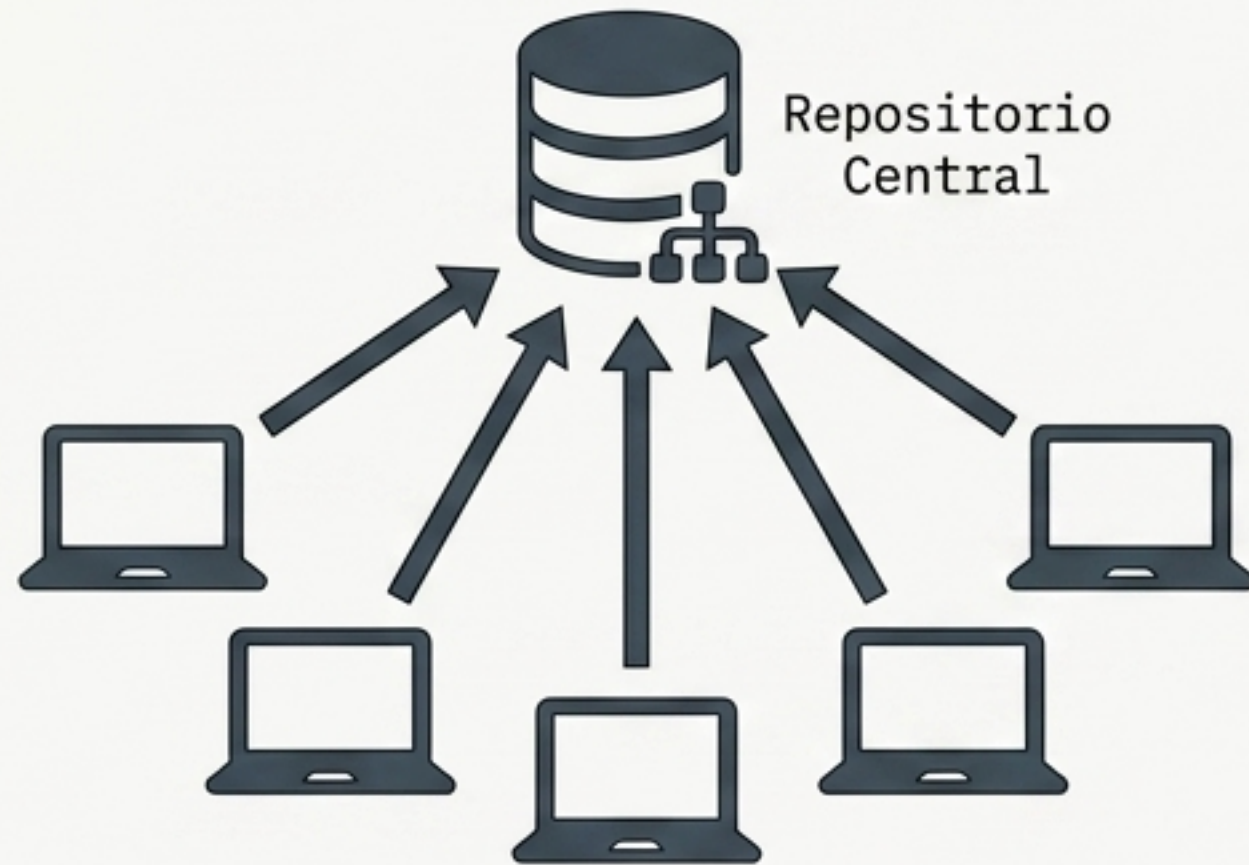
¿Qué es un Repositorio?

No es solo un servidor de archivos; es un servidor que guarda la evolución cronológica de cada documento.



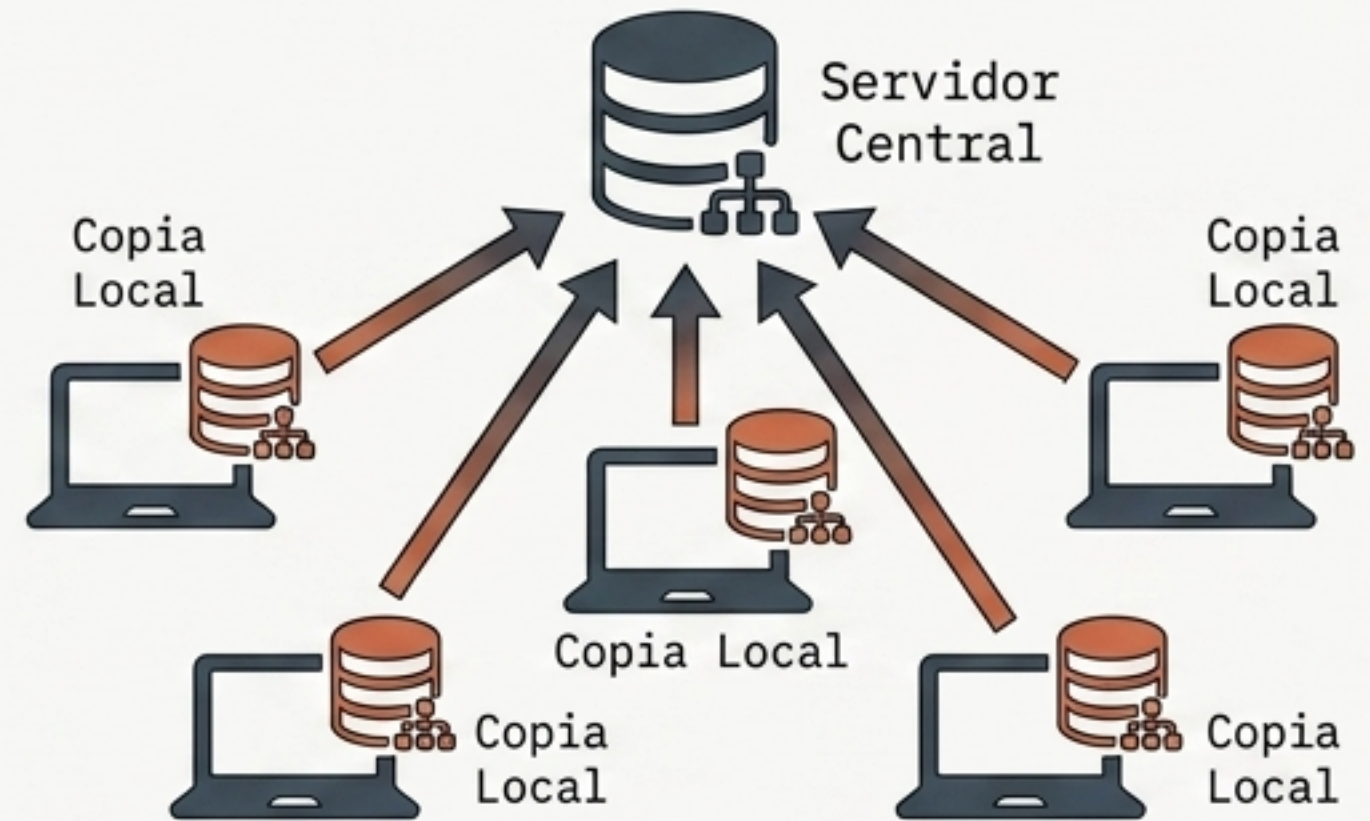
El Gran Debate Arquitectónico: Centralizado vs. Distribuido

Modelo Centralizado



Un único repositorio central. Los clientes descargan solo la parte del código necesaria para trabajar. Dependencia total del servidor.

Modelo Distribuido

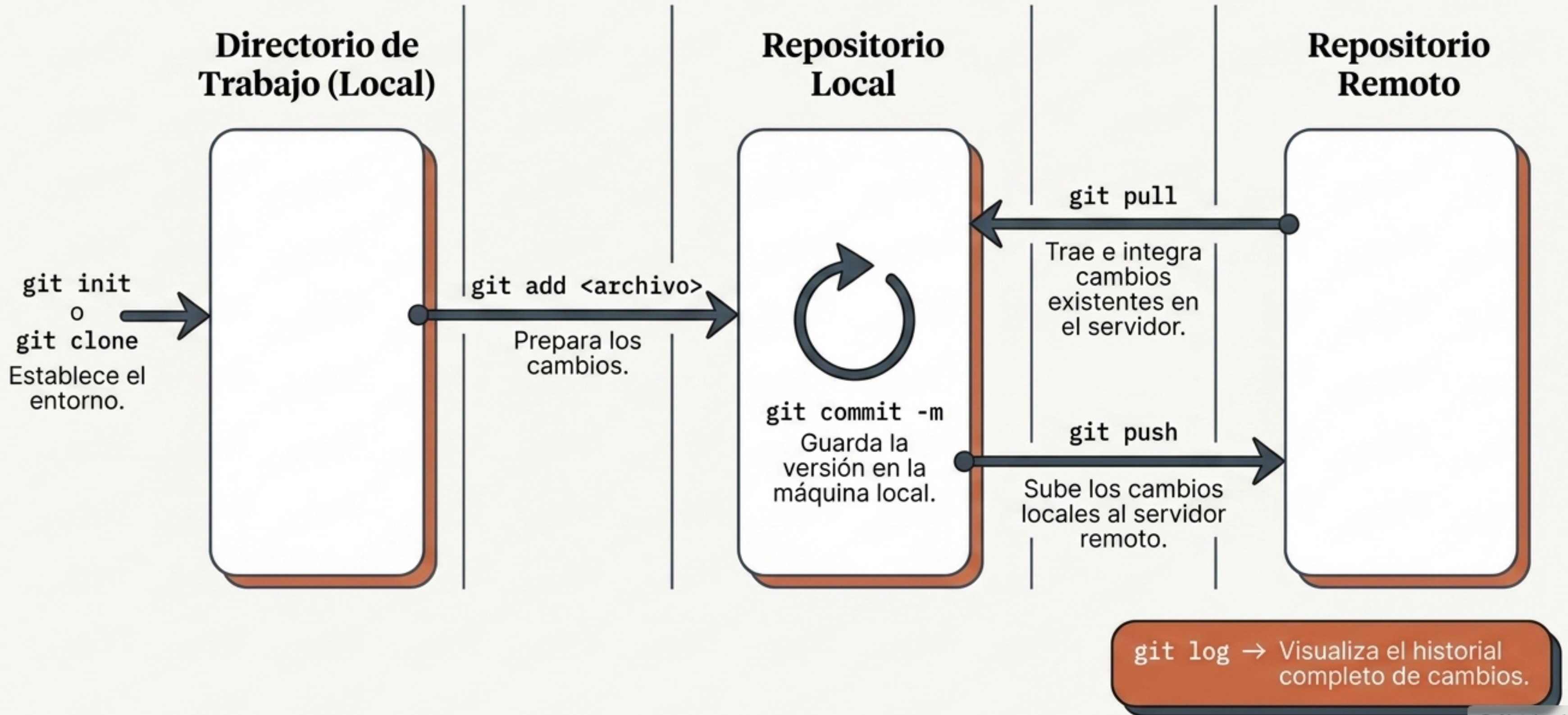


Cada usuario posee una copia local completa de todo el repositorio y su historial. Permite trabajar offline y cambiar entre versiones localmente.

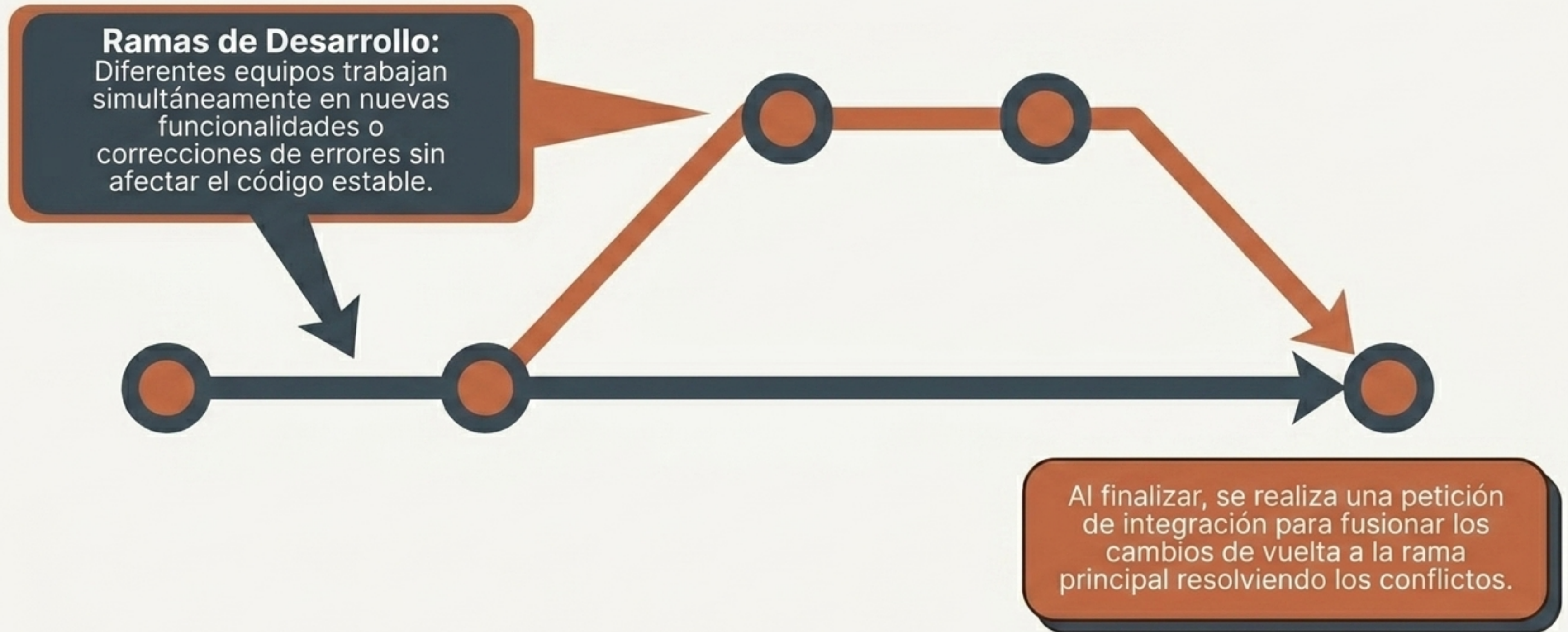
Git: El Estándar Distribuido Moderno



El Flujo de Trabajo en Git

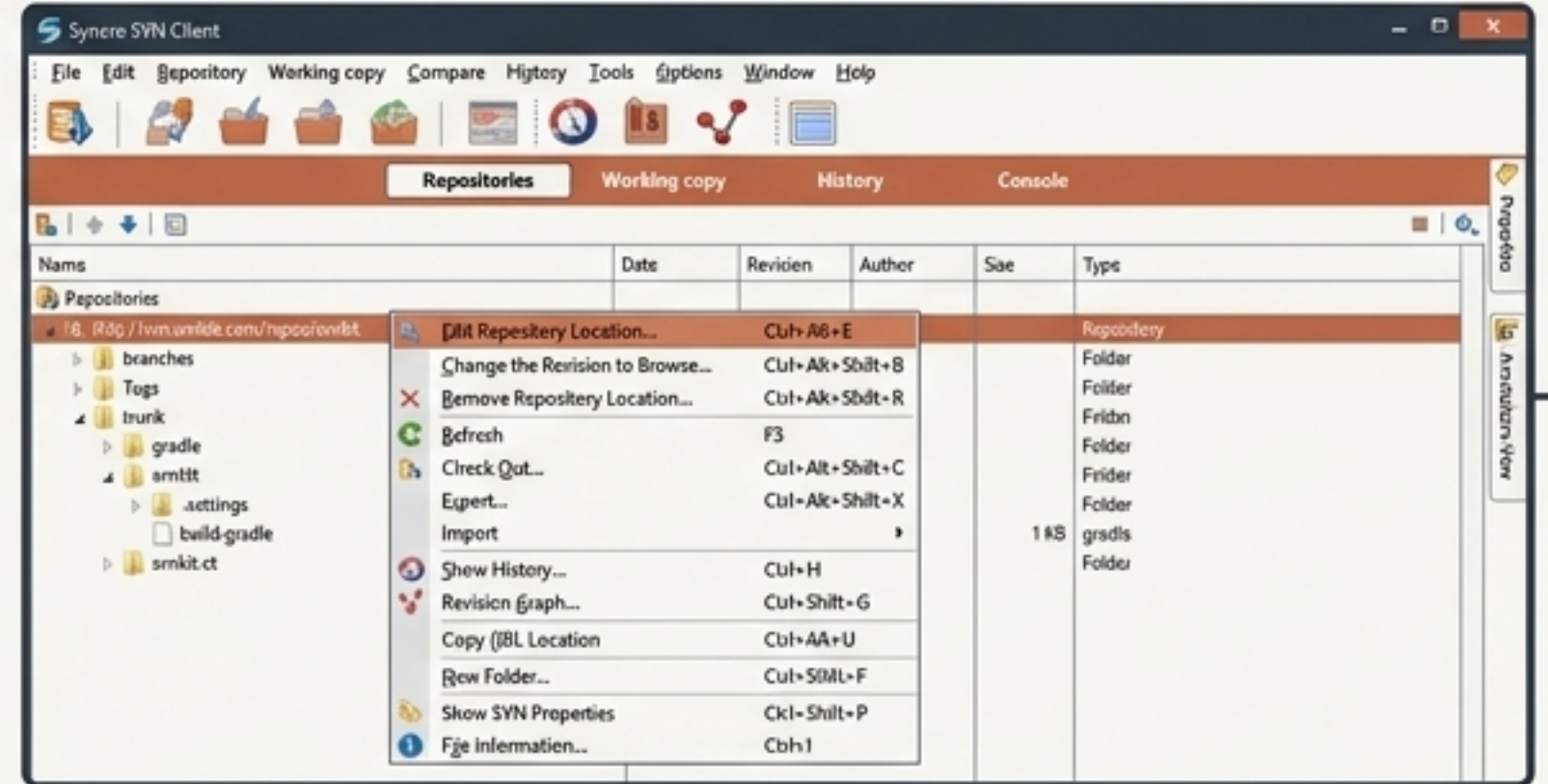


Aislamiento y Paralelismo: Ramas en Git



SVN (Apache Subversion): El Enfoque Centralizado

Un sistema veterano pero robusto, enfocado en el control absoluto desde un servidor único.



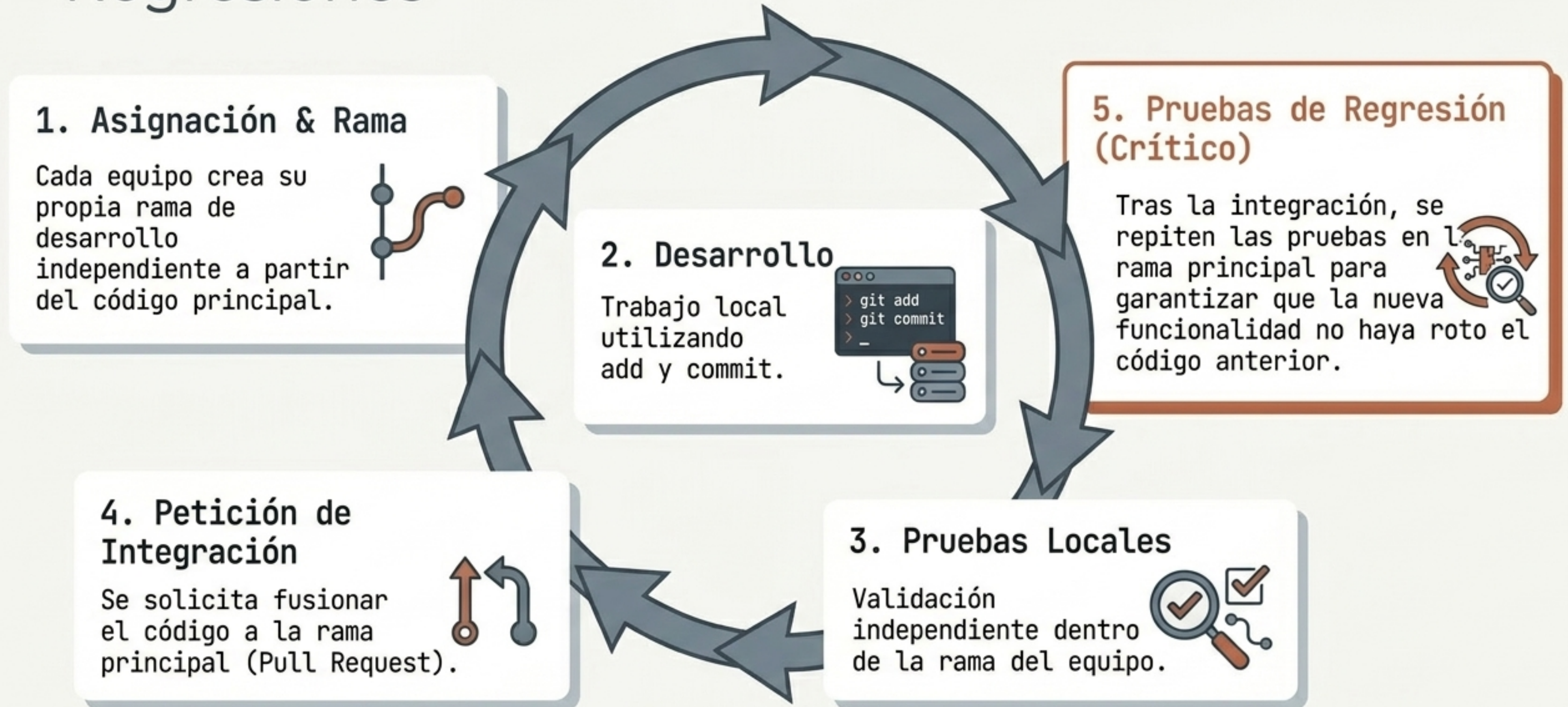
Ventajas: Bloqueo explícito de archivos, creación de enlaces externos y seguimiento estricto de modificaciones paso a paso.

Desventajas: Gestión compleja al renombrar archivos, menor soporte nativo en sistemas operativos modernos y dependencia absoluta de la conexión al servidor.

Matriz de Diagnóstico: ¿Git o SVN?

Escenario del Proyecto	Herramienta Recomendada
Contexto: Máxima seguridad requerida, prohibición de dispersar el código completo entre los desarrolladores, control estricto.	 SVN (Centralizado). Los usuarios solo descargan los fragmentos de código estrictamente necesarios.
Contexto: Equipo multidisciplinar grande, máxima velocidad de desarrollo, necesidad constante de crear múltiples ramas e integrarlas.	 Git (Distribuido). Alta productividad, copias locales que evitan cuellos de botella y facilidad para integraciones asíncronas.

Flujo de Integración Ideal y Prevención de Regresiones



Síntesis Ejecutiva

01

La Arquitectura Dicta el Flujo. Los sistemas distribuidos (**Git**) priorizan la agilidad y el trabajo offline, mientras que los centralizados (**SVN**) priorizan la seguridad y el control de acceso.

02

El Dominio de la Herramienta es Obligatorio. Independientemente de si se usa interfaz gráfica o línea de comandos, entender la lógica detrás de los repositorios y los *commits* es vital para no perder código.

03

Aislamiento y Regresión. Utilizar **ramas** protege el código estable. Realizar **pruebas exhaustivas** después de cada integración es el único método para evitar fallos en cadena.